

PROJECT REPORT

This end-to-end project report provides a comprehensive breakdown of the **Customer Shopping Behaviour Analysis** workflow, detailing each technical step from initial data cleaning to final visual intelligence.

1. Phase I: Data Engineering & Transformation (Python)

The first phase involved preparing the raw dataset using a Jupyter Notebook (Customerbehaviour.ipynb) to ensure data integrity and create meaningful features for analysis.

- **Initial Inspection & Cleaning:** * The dataset, comprising 3,900 records, was loaded into a Pandas DataFrame.
 - **Handling Missing Values:** The review_rating column had 37 missing values. These were handled by calculating the **median rating per product category** and filling the nulls accordingly, ensuring that ratings remained contextually accurate to the item type.
 - **Standardization:** To ensure compatibility with MySQL, all column names were converted to lowercase and spaces were replaced with underscores (e.g., Category became category).
- **Feature Engineering:**
 - **Age Segmentation:** A new age_group column was created using the qcut function, categorizing customers into four distinct life stages: **Young Adult, Adult, Middle-aged, and Senior**.
 - **Numerical Frequency Mapping:** Text-based purchase frequencies (e.g., "Weekly," "Annually") were mapped to a numerical purchase_frequency_days column to allow for quantitative time-based analysis (e.g., Weekly = 7 days).
- **Refinement:** A logical check confirmed that discount_applied and promo_code_used were identical. To reduce redundancy, the promo_code_used column was removed.

2. Phase II: Database Management & SQL Analytics

After preprocessing, the data was migrated to a MySQL environment to solve specific business problems through relational querying.

- **MySQL Integration:** Using sqlalchemy and pymysql, a connection was established to a local database named customer_behaviour, and the cleaned DataFrame was pushed into a table titled customer.
- **Analytical Problems Solved:**
 - **Gender-Based Revenue:** Calculated total spending by Male vs. Female shoppers to identify the primary revenue driver.
 - **High-Value Discount Users:** Identified customers who used discounts but still paid more than the average price, highlighting a high-spending "bargain-hunter" segment.
 - **Product Performance:** Ranked the top 5 products by average review rating and identified the **Top 3 products per category** using SQL window functions (ROW_NUMBER).

- **Shipping & Subscription Impact:** Compared average spend between **Standard and Express shipping**, and analyzed if **subscribers** generated more total revenue than non-subscribers.
- **Loyalty Segmentation:** Categorized the customer base into **New** (1 purchase), **Returning** (2–10), and **Loyal**(10+) based on their previous purchase history.

3. Phase III: Business Intelligence & Visualization (Power BI)

The final stage focused on translating the raw SQL results into an interactive Power BI dashboard (dashboard2.pbix) for stakeholders.

- **Executive KPIs:** Displayed high-level metrics including Total Revenue, Average Order Value (AOV), and Total Customer Count.
- **Behavioral Insights:** * A breakdown of revenue contribution by **Age Group** and **Location** was visualized to show which demographics are the most profitable.
 - A comparative analysis of **Subscribers vs. Non-Subscribers** demonstrated the financial impact of the loyalty program.
- **Product Strategy:** Visualized the "Discount Rate" per item to see which products rely most heavily on promotions to drive sales.

Conclusion & Recommendations

- **Focus on Loyalty:** The analysis identified a large "Returning" segment; the business should implement targeted email marketing to convert these users into "Loyal" customers.
- **Shipping Optimization:** Since shipping type impacts average spend, offering "Express" shipping incentives for high-value items could further increase revenue.
- **Stocking Decisions:** Inventory should be prioritized for the top 3 items in each category identified during the SQL analysis phase.

OUTPUT IMAGES

Step 1:

```
import pandas as pd
df = pd.read_csv('customer_shopping_behavior.csv')
```

```
df.head()
df.info()
```

```
<class 'pandas.DataFrame'>
RangeIndex: 3900 entries, 0 to 3899
Data columns (total 18 columns):
#   Column                                Non-Null Count  Dtype
---  -
0   Customer ID                          3900 non-null   int64
1   Age                                  3900 non-null   int64
2   Gender                              3900 non-null   str
3   Item Purchased                      3900 non-null   str
4   Category                            3900 non-null   str
5   Purchase Amount (USD)               3900 non-null   int64
6   Location                            3900 non-null   str
7   Size                                3900 non-null   str
8   Color                               3900 non-null   str
9   Season                              3900 non-null   str
10  Review Rating                       3863 non-null   float64
11  Subscription Status                 3900 non-null   str
12  Shipping Type                      3900 non-null   str
13  Discount Applied                   3900 non-null   str
14  Promo Code Used                    3900 non-null   str
15  Previous Purchases                 3900 non-null   int64
16  Payment Method                     3900 non-null   str
17  Frequency of Purchases              3900 non-null   str
dtypes: float64(1), int64(4), str(13)
memory usage: 548.6 KB
```

Step 2:

```
[3]: df.isnull().sum()
```

```
[3]: Customer ID          0
Age                    0
Gender                 0
Item Purchased         0
Category               0
Purchase Amount (USD)  0
Location               0
Size                   0
Color                  0
Season                 0
Review Rating          37
Subscription Status    0
Shipping Type          0
Discount Applied       0
Promo Code Used        0
Previous Purchases     0
Payment Method         0
Frequency of Purchases 0
dtype: int64
```

```
[4]: df['Review Rating'] = df.groupby('Category')['Review Rating'].transform(lambda x: x.fillna(x.median()))
df.isnull().sum()
```

```
[4]: Customer ID          0
Age                    0
Gender                 0
Item Purchased         0
Category               0
Purchase Amount (USD)  0
Location               0
Size                   0
Color                  0
Season                 0
Review Rating          0
Subscription Status    0
Shipping Type          0
Discount Applied       0
Promo Code Used        0
Previous Purchases     0
```

Step 3:

```
[5]: df.columns = df.columns.str.lower()
df.columns = df.columns.str.replace(' ','_')
df = df.rename(columns = {'purchase_amount_(usd)': 'purchase_amount'})

[19]: df.columns

[19]: Index(['customer_id', 'age', 'gender', 'item_purchased', 'category',
        'purchase_amount', 'location', 'size', 'color', 'season',
        'review_rating', 'subscription_status', 'shipping_type',
        'discount_applied', 'promo_code_used', 'previous_purchases',
        'payment_method', 'frequency_of_purchases', 'age_group',
        'purchase_frequency_days'],
        dtype='str')
```

Step 4:

```
[6]: #create a new column age group

labels = ['Young Adult','Adult','Middle-aged','Senior']
df['age_group'] = pd.qcut(df['age'],q=4,labels = labels)

[7]: df[['age', 'age_group']].head(10)

[7]:   age  age_group
0   55  Middle-aged
1   19  Young Adult
2   50  Middle-aged
3   21  Young Adult
4   45  Middle-aged
5   46  Middle-aged
6   63    Senior
7   27  Young Adult
8   26  Young Adult
9   57  Middle-aged
```

Step 5:

```
[8]: frequency_mapping = {
    'Fortnightly':14,
    'Weekly': 7,
    'Monthly': 30,
    'Quarterly': 90,
    'Bi-Weekly': 14,
    'Annually':365,
    'Every 3 Months': 90
}
df['purchase_frequency_days'] = df['frequency_of_purchases'].map(frequency_mapping)

[9]: (df['discount_applied']== df['promo_code_used']).all()

[9]: np.True_
```

Step 6:

```
[10]: df.drop('promo_code_used', axis = 1)

[10]:
```

	customer_id	age	gender	item_purchased	category	purchase_amount	location	size	color	season	review_rating	subscription_status	s
	0	1	55	Male	Blouse	Clothing	53	Kentucky	L	Gray	Winter	3.1	Yes
	1	2	19	Male	Sweater	Clothing	64	Maine	L	Maroon	Winter	3.1	Yes
	2	3	50	Male	Jeans	Clothing	73	Massachusetts	S	Maroon	Spring	3.1	Yes
	3	4	21	Male	Sandals	Footwear	90	Rhode Island	M	Maroon	Spring	3.5	Yes
	4	5	45	Male	Blouse	Clothing	49	Oregon	M	Turquoise	Spring	2.7	Yes
	...	...	...	...	...	...	...	...	...	...	...	...	...
	3895	3896	40	Female	Hoodie	Clothing	28	Virginia	L	Turquoise	Summer	4.2	No
	3896	3897	52	Female	Backpack	Accessories	49	Iowa	L	White	Spring	4.5	No
	3897	3898	46	Female	Belt	Accessories	33	New Jersey	L	Green	Spring	2.9	No
	3898	3899	44	Female	Shoes	Footwear	77	Minnesota	S	Brown	Summer	3.8	No
	3899	3900	52	Female	Handbag	Accessories	81	California	M	Beige	Spring	3.1	No

3900 rows x 19 columns

Step 7:

```
[15]: df.columns

[15]: Index(['customer_id', 'age', 'gender', 'item_purchased', 'category',
        'purchase_amount', 'location', 'size', 'color', 'season',
        'review_rating', 'subscription_status', 'shipping_type',
        'discount_applied', 'promo_code_used', 'previous_purchases',
        'payment_method', 'frequency_of_purchases', 'age_group',
        'purchase_frequency_days'],
        dtype='str')
```

Step 8:

```
[24]: from sqlalchemy import create_engine
import pandas as pd

# 1. Update your MySQL credentials here
# Standard MySQL default port is 3306
username = "root"
password = "gauravyadav15"
host = "localhost"
port = "3306"
database = "customer_behaviour"

# 2. The connection string for MySQL
# Format: mysql+pymysql://user:password@host:port/database
conn_str = f"mysql+pymysql://{username}:{password}@127.0.0.1:{port}/{database}"
engine = create_engine(conn_str)

df = pd.read_csv('customer_shopping_behavior.csv')

# 3. Try to push the data
try:
    # Assuming 'df' is your shopping dataframe
    df.to_sql('customer', engine, if_exists='replace', index=False)
    df.columns = [col.lower().replace(' ', '_') for col in df.columns]

# Verify the change
print(df.columns)
print("✅ Success! Your data is now in MySQL.")
except Exception as e:
    print(f"❌ Connection Error: {e}")

✅ Success! Your data is now in MySQL.
```

MYSQL QUERIES AND PROBLEM SOLVED QUESTION

1.What is total revenue generated by male vs female

select gender, sum(purchase_amount) as total_revenue_generated from customer group by gender;

2.which customers used discount but paid more than avg price

select * from customer; select customer_id,purchase_amount from customer where discount_applied = "Yes" and purchase_amount >= (select avg(purchase_amount) from customer);

3.which are the top 5 products with the highest avg review rating

select item_purchased, round(avg(review_rating),2) as "Avg product rating" from customer group by item_purchased order by avg(review_rating) desc limit 5;

4.compare the avg purchase amount between standard and express shipping

select round(shipping_type,2),avg(purchase_amount) from customer where shipping_type in ('Standard','Express') group by shipping_type ;

5.Do the subscriber customers spend more? compare average spend and total revenue between subscribers and non-subscribers

select subscription_status, count(customer_id) as total_customers, round(avg(purchase_amount),2) as avg_spend, round(sum(purchase_amount),2) as total_revenue from customer group by subscription_status order by total_revenue, avg_spend desc;

6.which 5 products have the highest percentage of purchases with discounts applied?

select item_purchased, round(sum(case when discount_applied = 'Yes' THEN 1 ELSE 0 end)/count(*) * 100,2) as discount_rate from customer group by item_purchased order by discount_rate desc limit 5;

7. Segment customers into New, Returning, and loyal based on their total -- number of previous purchase, and show the count of each segment

with customer_type as (select customer_id, previous_purchases, case when previous_purchases = 1 then 'Now' when previous_purchases between 2 and 10 then 'Returning' else 'Loyal' end as customer_segment from customer)

select customer_segment, count(*) as "Number of customers" from customer_type group by customer_segment;

8. What are the top 3 most purchased products within each category

with item_counts as (select category, item_purchased, count(customer_id) as total_orders, row_number() over(partition by category order by count(customer_id) desc) as item_rank from customer group by category, item_purchased)

select item_rank, category, item_purchased, total_orders from item_counts where item_rank <= 3;

9. Are customers who are repeat buyers (more than 5 previous_purchases) also likely to subscribe

select subscription_status, count(customer_id) as repeat_buyers from customer where previous_purchases > 5 group by subscription_status;

10. What is the revenue contribution of each row group?

select age, sum(purchase_amount) as total_revenue from customer group by age order by total_revenue desc;